



An Application-oblivious Memory Scheduling System for DNN Accelerators

JIANSONG LI, Huawei Galois Lab, China

XUEYING WANG, XIAOBING CHEN, and GUANGLI LI, Institute of Computing Technology, Chinese Academy of Sciences and University of Chinese Academy of Sciences, China

XIAO DONG, NVIDIA Corporation, China

PENG ZHAO and XIANZHI YU, Huawei 2012 Lab, China

YONGXIN YANG, WEI CAO, LEI LIU, and XIAOBING FENG, Institute of Computing Technology, Chinese Academy of Sciences and University of Chinese Academy of Sciences, China

Deep Neural Networks (DNNs) tend to go deeper and wider, which poses a significant challenge to the training of DNNs, due to the limited memory capacity of DNN accelerators. Existing solutions for memory-efficient DNN training are densely coupled with the application features of DNN workloads, e.g., layer structures or computational graphs of DNNs are necessary for these solutions. This would result in weak versatility for DNNs with sophisticated layer structures or complicated computation graphs. These schemes usually need to be re-implemented or re-adapted due to the new layer structures or the unusual operators in the computational graphs introduced by these DNNs.

In this article, we review the memory pressure issues of DNN training from the perspective of runtime systems and model the memory access behaviors of DNN workloads. We identify the *iterative*, *regularity*, and *extremalization* properties of memory access patterns for DNN workloads. Based on these observations, we propose AppObMem, an application-oblivious memory scheduling system. AppObMem automatically traces the memory behaviors of DNN workloads and schedules the memory swapping to reduce the memory pressure of the device accelerators without the perception of high-level information of layer structures or computation graphs. Evaluations on a variety of DNN models show that, AppObMem obtains 40–60% memory savings with acceptable performance loss. AppObMem is also competitive with other open sourced SOTA schemes.

CCS Concepts: • **Software and Runtime System** → *Memory Management*; • **Computing methodologies** → *Neural networks*;

Additional Key Words and Phrases: Deep learning, memory scheduling, runtime system, DNN accelerators

This work is supported in part by the National Natural Science Foundation of China (61872043).

Jiansong Li and Xueying Wang contributed equally to this work. This work was initiated and conducted while Jiansong Li was a student at ICT.

Authors' addresses: J. Li, Huawei Galois Lab, Beijing, China; email: jsonlee@whu.edu.cn; X. Wang, X. Chen, G. Li (corresponding author), Y. Yang, W. Cao, L. Liu, and X. Feng, State Key Laboratory of Computer Architecture, Institute of Computing Technology, Chinese Academy of Sciences, No. 6 Kexueyuan South Road Zhongguancun, Haidian District, Beijing, China, 100190 and School of Computer Science and Technology, University of Chinese Academy of Sciences, No. 19(A) Yuquan Road, Shijingshan District, Beijing, China, 100049; emails: {wangxueying, chenxiaobing, liguangli, yangyongxin, caowei, liulei, fxb}@ict.ac.cn; X. Dong, NVIDIA Corporation, Shanghai, China; email: xiaod@nvidia.com; P. Zhao and X. Yu, Huawei 2012 Lab, Beijing, China; emails: {zhaopeng71, yuxianzhi}@huawei.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.

1544-3566/2022/09-ART47 \$15.00

<https://doi.org/10.1145/3535355>

ACM Reference format:

Jiansong Li, Xueying Wang, Xiaobing Chen, Guangli Li, Xiao Dong, Peng Zhao, Xianzhi Yu, Yongxin Yang, Wei Cao, Lei Liu, and Xiaobing Feng. 2022. An Application-oblivious Memory Scheduling System for DNN Accelerators. *ACM Trans. Arch. Code Optim.* 19, 4, Article 47 (September 2022), 26 pages.
<https://doi.org/10.1145/3535355>

1 INTRODUCTION

Machine learning (ML), especially **deep neural networks (DNNs)** techniques, have been pervasive tools in various application fields, including computer vision and natural language understanding [11]. To achieve higher prediction accuracy, neural network structures become deeper and wider [33]. A technical report from OpenAI has shown that the computation and memory requirements of DNN workloads are doubled every 3.4 months in recent years [48]. As such, DNN workloads usually consume very large resources, due to their computations on the increasing numerous neural network parameters.

Contemporary execution environment of DNN workloads is usually heterogeneous, where the host CPUs are used for task scheduling and partial operations (e.g., data pre-processing or post-processing) and the device accelerators are responsible for accelerating computing-intensive tasks (e.g., matrix multiplication and convolution). Representative device accelerators, including GPUs and ASIC-based DNN accelerators [10, 30], have shown significant performance improvement over traditional CPUs for DNN workloads. On heterogeneous platforms, before execution of the device accelerators, the data required need to be transferred to the device memory. However, the memory capacity of device accelerators is usually far less than the capacity of host CPUs, resulting in a memory limitation when utilizing accelerators. For instance, the memory capacity of Huawei Ascend910 NPU is 32 GB [26], and the memory capacity of Nvidia Ampere GPUs is 40 GB [47]. Unfortunately, with such memory capacities it is hard to satisfy the demands of emerging DNN workloads. For example, Inception-v4 [60], which is a widely used DNN structure, requests up to 45 GB of memory to keep the entire DNN model on the device during training [69]. Although model parallelism [16] is a practical approach to reduce the memory pressure issue, modification of the original code implementation and considerations for algorithm details are indispensable. The major efforts of ML practitioners ought to be devoted to their algorithm and model architecture design rather than the system implementations of these DNNs. As shown in Figure 1(a), the limited memory capacity of DNN accelerators becomes a hurdle for the researchers in the DNN community to further explore deeper and wider DNNs for higher accuracy.

To work around the memory capacity bottlenecks, various techniques have been proposed, including but not limited to model compression [13, 17, 21, 37, 43], memory sharing [7, 8, 29, 51, 66], re-computing [8, 20, 50, 64, 65], or swapping-based approaches [9, 15, 24, 34, 50, 55, 56, 65, 69, 70]. However, these approaches have their own limitations. Those model compression techniques, like pruning and quantization, tend to result in accuracy drop, which is nonnegligible in the DNN domain. To make up for the accuracy loss, a lot of fine-tuning efforts are required, e.g., re-training. State-of-the-art memory footprint reduction schemes, like memory sharing, re-computing, and memory swapping-based approaches, usually tightly depend on the application features of DNN workloads. From the source of scheme design, these solutions usually take the layer structures or the computational graphs of DNN workloads as their input. Inside their cost models, they need to enumerate the layer structures of the given DNNs or the operators in the computational graphs to determine the memory scheduling plan. Unfortunately, DNNs evolve at an alarming rate. Many new layer structures and operators are proposed every year, for example, the new dual GRU layers introduced by FeatherWave [62] and the convolution operators with cheaper additions

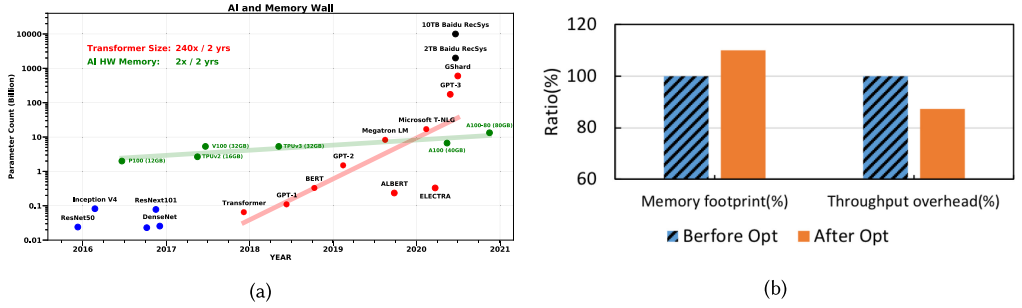


Fig. 1. Development trend of DNN model size and device memory capacity of DNN accelerators in recent years [19] (Figure 1(a)). For the Gradient Boosting Decision Tree (GBDT) model, TF-LMS [34] can even enlarge the device memory pressure if we enable its memory scheduling optimization (Figure 1(b)).

introduced by AdderNet [6]. These application-densely coupled solutions are not friendly for those new DNNs with sophisticated layer structures or complicated computation graphs and would result in weak versatility. Let us take the typical memory swapping-based TF-LMS [34] as an example. As shown in Figure 1(b), for the general machine learning **Gradient Boosting Decision Tree (GBDT)** model [52] with complicated computation graphs, TF-LMS could even enlarge the memory pressures and introduce nonnegligible runtime overhead if we enable its memory scheduling optimization. This is mainly because the memory scheduling optimization of TF-LMS heavily relies on the layer structures and computational graphs of DNN workloads with typical convolutional operators, which are rare in the application features of GBDT.

Toward the issues above, the question is whether we can propose a more versatile memory-efficient scheme without perceiving the application features (e.g., layer structures or computational graphs of DNNs)? We seek to answer the question by carefully characterizing the memory behaviors of DNN workloads. We model and pinpoint the memory behaviors of each device memory block of the DNN accelerators by instrumenting the memory allocators of the runtime system. Our characterizing results show that the memory access patterns of each memory block follow a *regular*, *iterative*, and *extremal* fashion. Furthermore, we also observe that the latency of swapping data back and forth between host CPUs and the device accelerators is easy to model, which can be predicted with a linear expression. Motivated by these observations, and inspired by the swapping-based memory footprint reduction schemes, we design and implement AppObMem, an application-oblivious memory scheduling system for DNN accelerators. Unlike existing solutions that require the layer structures or computational graphs of DNNs as input, AppObMem only needs the host-device swapping bandwidth information. It automatically traces the memory access patterns of DNN workloads and builds the cost model to determine which memory block to swap-out or swap-in at the right time. Considering the unique regular and iterative feature of DNN workloads, AppObMem automatically builds the cost model from the memory access behaviors of the previous iterations. To reduce the runtime overhead, AppObMem deploys the retained cost model for the subsequent rounds of iterations. Experimental results illustrate that AppObMem effectively covers most DNNs (including those typical linear and non-linear DNNs, e.g., CNNs, RNNs, BERT, and machine learning models) and leads to memory-efficient solutions with negligible throughput loss. AppObMem reviews the memory footprint reduction problem from the perspective of the underlying runtime systems, i.e., the memory access behaviors of DNN workloads, which is independent from the high-level application layer structures and computational graphs. This decouples the memory footprint reduction schemes from the frequently updated application features of DNN workloads. Our main contributions are as follows:

Table 1. Memory-related Programming APIs for DNN Accelerators

Platforms	Malloc	Free	Memcpy	Async Execution
Nvidia GPU	cudaMalloc(...), cudaMallocHost(...)	cudaFree(...), cudaFreeHost(...)	cudaMemcpy(...), cudaMemcpyAsync(...)	cudaStream_t, cudaStreamSynchronize(...)
Cambricon MLU	cnrtMalloc(...), cnrtMallocHost(...)	cnrtFree(...), cnrtFreeHost(...)	cnrtMemcpy(...), cnrtMemcpyAsync(...)	cnrtQueue_t, cnrtSyncQueue(...)
Huawei Ascend910	rtMalloc(...), rtMallocHost(...)	rtFree(...), rtFreeHost(...)	rtMemcpy(...), rtMemcpyAsync(...)	rtStream_t, rtStreamSynchronize(...)

Note that for more details about these APIs, please refer to the programming manuals of the corresponding platform [5, 27, 45].

- We analyze the limitations of existing memory scheduling schemes and identify the *iterative*, *regularity*, and *extremalization* properties of memory access patterns for DNN workloads, thereby motivating the design of AppObMem.
- We propose AppObMem, a versatile memory scheduling system without the perception of high-level application features, such as DNN layer structures and computational graphs. AppObMem automatically traces the memory behaviors of DNN workloads and schedules the memory swapping to reduce the memory pressure of the device accelerators.
- We provide a detailed evaluation of AppObMem with performance comparisons to several SOTA open sourced schemes on the typical linear and non-linear DNNs, e.g., CNNs, RNNs, BERT, and machine learning models. AppObMem obtains 40–60% memory savings with acceptable performance loss. It is also competitive with other open sourced SOTA schemes.

The rest of this article is organized as follows: Section 2 reviews the background and related work. Section 3 presents the case study and the motivation of this article. The design principles and implementation details of AppObMem are illustrated in Section 4. The experimental results of AppObMem are presented in Section 5. Sections 6 and 7 discuss the limitations of AppObMem and conclude, respectively.

2 BACKGROUND AND RELATED WORK

2.1 Heterogeneous Environment and DNN Workloads

The running environment of DNN workloads is usually a heterogeneous system, consisting of host CPUs and device accelerators. The device accelerators are usually attached to the host CPUs through the high-performance bus, e.g., PCIe and NVLink for GPUs. Host CPUs usually play the role of control, e.g., launch kernel and thread scheduling. The device accelerators are responsible for the calculations of DNN-related tasks, e.g., matrix multiplication, convolution, pooling, and activation operations. Before execution of the device accelerators, the input data needs to be residing on the device memory. Vendors of device accelerators usually provide an SDK with high-level programming APIs to manage the device memory and execution orders and swap data back and forth between the host CPUs and device accelerators. For example, as shown in Table 1, CUDA programmers can use *cudaMalloc/cudaFree* to malloc/free the device memory blocks on GPUs and use *cudaMemcpy* to swap data between CPUs and GPUs. Vendors also provide asynchronous APIs to overlap the I/O and execution, e.g., *cudaStream_t*.

Modern DNNs usually consist of numerous layers. These layers are composed together by a complicated *linear* or *non-linear* topology. For example, AlexNet has 24 layers that are organized in a linear fashion; ResNet50 has 228 layers, consisting of 50 residual blocks [22]. These blocks are composed together in non-linear topology, where there are many forks (one-to-many) and joins (many-to-one) layer structures in terms of topology. During the inference and training of DNN workloads, DNNs are usually expressed as computation graphs, where the nodes represent

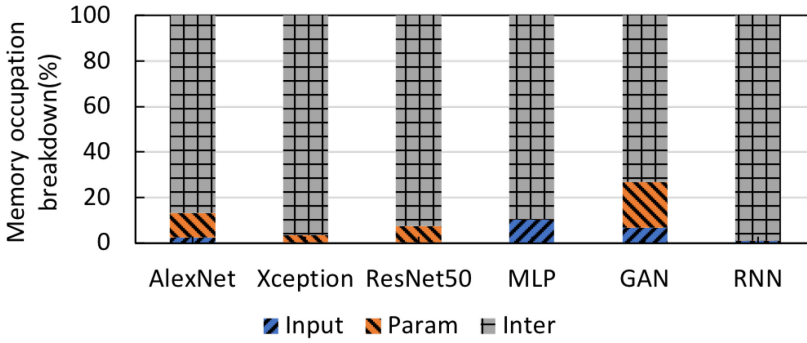


Fig. 2. Memory occupation breakdown of typical DNN workloads.

basic operations (namely operators, e.g., convolution, pooling, activation) and the edges refer to data consumed or produced by these operations. From the perspective of the storage content of device memory blocks, the memory consumption of DNN workloads falls into three categories as follows:

- **Input Data.** During inference and training, the input validation samples are usually processed in a batching fashion. Before the execution of the device accelerators, these batching samples need to be residing on the device memory. The memory consumption of input data is usually proportional to batch size.
- **Parameters.** In the feed-forward inference stage, the parameters (also known as weights) are usually read-only, whose values determine how much influence the validation samples will have on the output. The training stage consists of two phases, i.e., forward and backward passes. Specifically, during the backward phase, the backward propagation algorithm is to propagate the error and calculates the gradient of the loss function that can be applied to adjust the parameters toward improving the accuracy [35]. In the training stage, parameters are updated at the end of each iteration and used by the next iteration. The memory consumption of parameters is usually proportional to the numbers of layers (depth) and size of the layer (width) for the given DNNs;
- **Intermediate Results.** DNN workloads tend to produce three types of intermediate results: feature maps, gradient maps, and workspace. Feature maps are the intermediate results that are consumed by the next forward or backward layers. In the forward pass, the input feature maps are fed to the current layer in the forward direction through the DNN. Each hidden layer accepts the input data, processes it as per the activation function, and passes its output to the successive layer. At the end of the forward pass, the network obtains the loss and then computes gradients to update weights during the backward propagation. Feature maps generated in the forward propagation are usually residing on the device memory until they are used to compute the gradients. The overall memory usage will increase during the forward propagation. Gradient maps are the intermediate results that are generated in the backward pass and consumed by the dependent layers [35]. The workspace is the intra-layer storage to speed up the layer computation. For example, the workspace of convolution algorithms requires additional but temporary device memory to achieve better performance [12, 55].

Figure 2 presents the breakdown of memory footprint during training for different DNN workloads on the Nvidia Titan X Pascal GPU. We observe that, for most DNNs, the *parameters* only account for a small fraction of the total memory footprint. In training, the *intermediate results* are the primary contributor to the memory footprint.

2.2 Memory Footprint Reduction Schemes

Many approaches have been proposed to reduce the memory footprint of DNN workloads, including model compression, memory sharing, re-computation, and swapping-based solutions.

2.2.1 Model Compression. Typical model compression techniques include pruning [17, 21, 37, 38, 43] and quantization [11, 13, 28, 36]. Model pruning is based on the fact that there is a great deal of redundancy in parameters [42]. Pruning techniques try to prune weights without losing accuracy much. As a result, both performance and memory usage are improved. Alternatively, DNNs can also be compressed through quantization, i.e., by reducing the number of bits used to represent each parameter. Some research even exploited the use of binary numbers to boost the performance of DNN workloads, which also brings memory usage savings [13]. Our statistics in Figure 2 show that parameters only occupy little memory consumption for typical DNN workloads, indicating that weight pruning or quantization techniques are not efficient for reducing the memory footprint in training. Besides, these model compression techniques usually lead to accuracy loss, which is nonnegligible in the DNN domain.

2.2.2 Memory Sharing. Common memory sharing techniques fall into two categories, i.e., in-place sharing [7, 8] and memory reuse [29, 51, 66]. As the name suggests, in-place sharing is to reuse the same memory region for input and output. For example, for the ReLU activation layers, i.e., $y = \text{relu}(x)$, y and x can share the same memory block during computing. There are only a few in-place sharing opportunities in DNNs [8]. Another memory sharing technique is the cross-layer memory reuse, the most representative scheme is layer-fusion based solutions, like TASO [29]. These techniques try to replace the structured blocks in the original computation graph with some fused blocks through graph substitutions. There are also some reuse opportunities for those variables in the fused blocks whose lifetimes do not overlap. Layer-fusion techniques tend to remove the temporary variables across layers, thus reducing the memory footprint.

2.2.3 Output Re-Computation. Output re-computation schemes are based on the idea that trading computation for memory saving. These approaches discard some computing results when the device memory is insufficient and re-compute them when they are required [8, 20, 50, 64, 65]. For example, some intermediate results, e.g., certain feature maps generated in the forward propagation phase can be discarded until they are required in the backward propagation phase. Similar ideas have been adopted in BPTT [20], SuperNeurons [65], Capuchin [50], and KARMA [64]. All of them use an empirical cost model to selectively drop the feature maps, which are cheap to re-compute. The empirical cost model is densely coupled with the layer structure or semantics from the computation graphs of the DNN workloads. These approaches may incur explicit performance degradation due to the re-computation time for some layers. This approach may work well for those linear networks (no forks and joins in the layer structures of DNNs, e.g., AlexNet and VGG [59]), since there are no complex re-computation dependencies across the sequential layers of the linear DNNs. But for those non-linear DNNs with many forks and joins in their layer structures, the complicated execution dependencies between layers may introduce complex re-computation dependencies. For those new DNNs with sophisticated layer structures or complicated computation graphs, the situation can be even worsened.

2.2.4 Swapping-based Schemes. Another promising approach aiming to reduce the memory footprint of DNN workloads is to virtualize the memory usage of DNNs across the host and device memory [3, 9, 15, 24, 34, 50, 55, 56, 65, 69, 70]. Rather than always keeping data residing on the device memory, these swapping-based schemes adopt the idea of data offloading and prefetching. Data that are not in use are swapped out to the host memory, and they are swapped back into

Table 2. Comparisons of Related Approaches

Approaches	Representative Schemes	w/o App. Feat. as Input?	w/o Acc. Drop?	Transparent to End-Users?	Strong Versatility?
Model Compression	Pruning [17, 21, 37, 43]	✗	✗	○	○
	Quantization [11, 13, 28]	✗	✗	○	○
Memory Sharing	In-place Sharing [7, 8]	✗	✓	○	✗
	Cross-layer Reuse [29, 34, 51, 66]	✗	✓	○	✗
Output Re-Computation	BPTT [20]	✗	✓	○	✗
	SuperNeurons [65]	✗	✓	✗	✗
	Capuchin [50]	✗	✓	○	✗
	KARMA [64]	✗	✓	○	✗
Memory Swapping	moDNN [9]	✗	✓	✗	✗
	vDNN [55]	✗	✓	✗	✗
	vDNN++ [56]	✗	✓	✗	✗
	TF-LMS [34]	✗	✓	✗	✗
	SwapAdvisor [24]	✗	✓	○	✗
AppObMem (Ours)	—	✓	✓	✓	✓

Note that the application features include the layer structures and the high-level semantics from the computation graphs of DNN workloads. Symbols ✓: Yes; ✗: No; ○: Unknown.

the device memory right before they are required. To reduce the swapping overhead, the swapping in both directions should be hidden with the computations through separate asynchronous streams. In GeePS [15], the decision to swap which layer is made by the end-users. It requires the end-users to have a good understanding of the DNN workloads, including the memory and time consumption of each layer [70]. vDNN [55] takes a brute force policy, and it simply offloads the outputs of all layers or all convolution layers. vDNN++ [56] proposes an approximated memory pool to reduce the GPU memory fragmentation. Both SuperNeurons [65] and Capuchin [50] combine swapping with re-computation. The former considers memory swapping but restricts to swapping only convolution layers. The latter considers swapping at the granularity of tensors, which closely depends on the semantic information of computational graphs. Both of them require high-level semantics from the computation graphs of the DNN workloads. TF-LMS [34] proposes some formal graph rewriting rules to insert the swapping-in or swapping-out nodes at the computational graphs. SwapAdvisor [24] performs joint optimization along three dimensions based on the computation graphs of DNN workloads, including operator scheduling, memory allocation, and swapping decisions. It also proposes a custom-designed genetic algorithm-based heuristics to explore the vast search space.

As presented in Table 2, we highlight the limitations of existing schemes on reducing the memory pressure of DNN workloads. Model compression techniques may lead to accuracy drop, which is nonnegligible in the DNN domain. A lot of fine-tuning efforts are also required to make up for the accuracy loss. Existing memory sharing, output re-computation and memory swapping-based solutions require the high-level application features as input and some of them even require the intervention of end-users. They are not friendly for those DNNs with sophisticated layer structures or complicated computation graphs. It is desirable to propose a *strongly versatile* scheme that is *transparent* to the end-users *without affecting the accuracy* and is able to address the memory pressure issues of DNN workloads with *small runtime overhead*.

3 CASE STUDY AND MOTIVATION

A promising approach to address the device memory pressure issues of DNN workload without affecting accuracy is to swap data between the host and device memory during DNN computation. Several technical development trends confirm that memory swapping approaches are attractive. First, the host CPU memory is usually much larger and cheaper than device memory. Besides,

Table 3. Definition and Explanation of Notations Used by AppObMem

Notations	Explanation
W	Current DNN workload
A	Memory access actions, which can be <i>malloc</i> , <i>free</i> , <i>read</i> , <i>write</i>
B	Device memory block
P	Start pointer of current memory block
S	Size of current memory block
$\langle A, B, T \rangle$	During the execution of W , a memory access action A occurs to memory block B at the time of T
T	Current timestamp
ΔT	Access time interval between two adjacent memory accesses to the same memory block
H_k	Highwater mark of memory payload over the first k memory access requests
α_i	Contribution factor of device memory payload for the i -th memory access behavior, which can be -1 , 0 , or 1
$T_{sw-in}(s)$	Latency of copying s bytes data block from host to device
$T_{sw-out}(s)$	Latency of copying s bytes data block from device to host
$T_{valid}^{(k)}$	Valid time interval to swap the memory block of the k -th memory access
G_k	Memory-saving gain of swapping out the memory block B_k

the communication bandwidth between the host CPU and the device accelerators is sufficiently performant with the arrival of PCIe 5.0 [49] and the high-speed interconnect bus provided by the accelerator vendors, e.g., NVLink for Nvidia GPUs [24]. Like vDNN [55] and SwapAdvisor [24], AppObMem adopts the idea of data offloading and prefetching. However, compared with existing schemes, AppObMem works without perceiving the high-level application features as input, due to the unique memory behaviors of DNN workloads. We reveal these patterns through the case study, which motivates the design of AppObMem.

3.1 Modeling Memory Access Behaviors of DNN Workloads

During the execution of DNN workload W , the device memory of the DNN accelerator will be split into chunks of memory blocks B . The memory access behaviors related to the device memory blocks fall into four types of actions: allocation of device memory (*malloc*), deallocation of current memory block (*free*), or read or write for some memory block. To characterize the memory behaviors of DNN workloads, we define some terms in Table 3. For a given DNN workload W , assumed that the access action A of the device memory block B occurs at timestamp T during its execution, then this memory access behavior is characterized by the triple $\langle A, B, T \rangle$. The memory access action A can be *malloc*, *free*, *read*, or *write*, i.e., $A \in \{\text{malloc}, \text{free}, \text{read}, \text{write}\}$. Each device memory block B , starting from the memory address P , with block size S , is characterized by $B := \langle P, S \rangle$. For a certain device memory block B_i , its two adjacent memory access behavior are marked by $\langle A_k, B_i, T_k \rangle$ and $\langle A_{k+1}, B_i, T_{k+1} \rangle$, respectively. Then the access time interval (ΔT) between this two adjacent memory accesses to B_i is $T_{k+1} - T_k$, i.e., $\Delta T_k = T_{k+1} - T_k$.

3.2 Memory Behavior Patterns of DNN Workloads

To work around the memory challenge of DNN workloads, we carefully characterize the memory behaviors of DNN workloads by pinpointing the behaviors of each device memory block of the DNN accelerator via instrumenting the allocators of the runtime system. Suppose we want to train

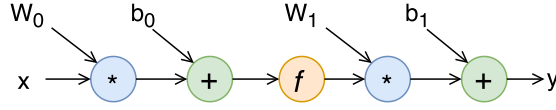


Fig. 3. Layer topology of the MLP testcase.

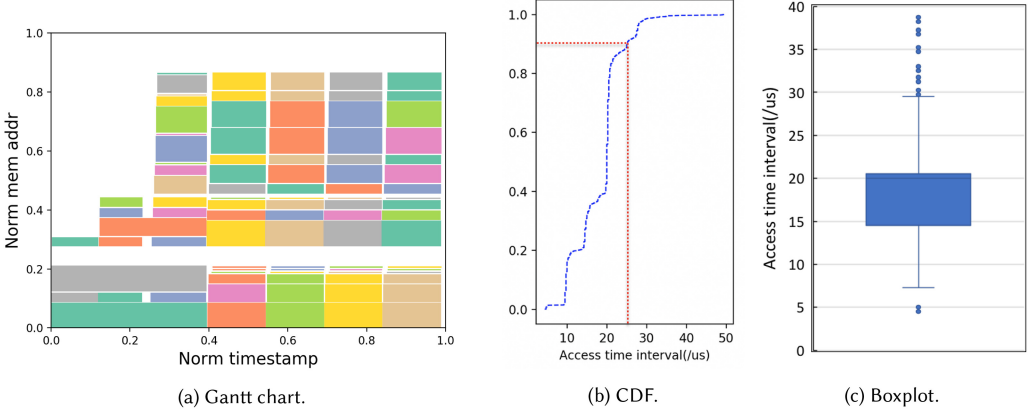


Fig. 4. Gantt chart of the first five iterations in MLP training (Figure 4(a)). Cumulative Distribution Function (CDF) and boxplot of the memory block access time intervals (ΔT s) in MLP training. Note that, to clearly show the distributions of memory behaviors, we discard some outliers with too much longer access time intervals when drawing CDF and boxplot.

a trivial two-layer MLP DNN [23], its topology is shown in Figure 3. The shape of w_0 is (2, 12288), b_0 : (12288), w_1 : (12288, 2), b_1 : (2).

We have drawn the Gantt chart of the first five rounds of iterations in the training of MLP. The x -axis of Figure 4(a) is the normalized timestamp, the y -axis is the normalized memory address of GPU. Each rectangle in Figure 4(a) represents the memory access behaviors to some memory block in the device memory of GPU. The width of the rectangle is the elapsed time from the allocation time to the free time of current memory block, which can be regarded as the lifetime of current memory block. The height of the rectangle denotes the size of current memory block. Obviously, we can see from Figure 4(a) that there are iterative memory access patterns in the first five rounds of iterations during the training process of MLP workload. Besides, the overlap between rectangles along the y -axis represents the overlap of the live ranges of different memory blocks. The blank space between two rectangles along the y -axis represents the memory fragments of GPU. We can conclude from Figure 4(a) that memory access patterns of DNN workloads present an iterative fashion.

Furthermore, we also draw the Cumulative Distribution Function chart of the access time intervals of memory blocks during the MLP training. The access time interval (ΔT in Table 3) is the elapsed time between two adjacent memory access to the same memory block. As illustrated in the boxplot of Figure 4(c), ΔT s of most memory behaviors are concentrated between 15 and 20 μs , and their distributions are very stable. Besides, Figure 4(b) shows that ΔT s of 90% of the memory behaviors are less than 25 μs . Suppose we want to reduce the device memory pressure by swapping data back and forth between the host CPUs and the device GPUs, like vDNN [55] and SwapAdvisor [24]. Assume the memory bandwidth from device GPU to host CPU is B_{d2h} and that of host to device is B_{h2d} . Then the maximum value of memory swapping size S without sacrificing

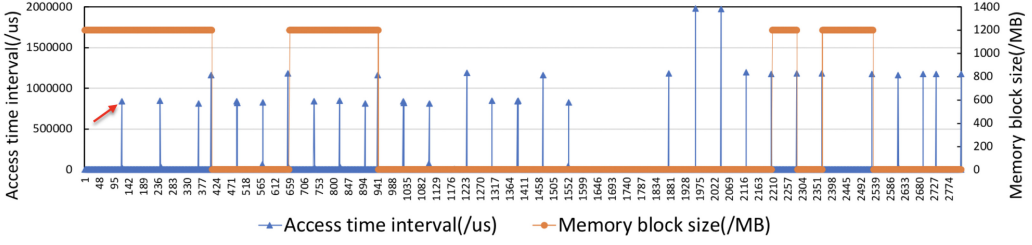


Fig. 5. Pairwise access time intervals (ΔT s) and the corresponding memory block size during MLP training.

the runtime performance should satisfy $\frac{S}{B_{d2h}} + \frac{S}{B_{h2d}} \leq \Delta T$, so we have the following equation:

$$S \leq \frac{\Delta T}{1/B_{d2h} + 1/B_{h2d}}. \quad (1)$$

We have measured the *memcpy* bandwidth between the device GPU and host CPU with the *bandwidthTest* tool from CUDA SDK samples [46]. The pinned memory transfer bandwidth from host to the device is 6.3 GB/s and from device to host is 6.4 GB/s. So the maximum value of memory swapping size without sacrificing the runtime performance is about 79.37 KB (by Equation (1), $S \leq \frac{25 \mu s}{1/6.4 \text{ GB/s} + 1/6.3 \text{ GB/s}} = 79.37 \text{ KB}$). This is just a drop in the bucket for reducing the memory pressure of MLP training. It seems that memory swapping is promising less.

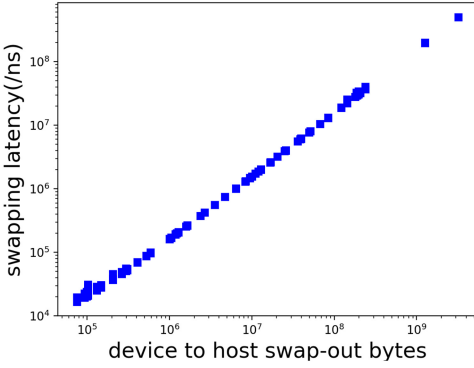
However, we also draw the detailed memory behaviors including ΔT s along with the corresponding memory block size in MLP training. As shown in Figure 5, access time intervals of most memory access behaviors are negligible, which is consistent with the observation from Figure 4. However, there are some outliers whose ΔT is larger than 0.8 seconds, and their corresponding memory block size is larger than 600 MB. For example, ΔT and the corresponding memory block size of the red marked outlier are 840211 μs and 1200 MB, respectively. In this case, the boundary data swapping size without sacrificing the runtime performance between the host and device is about 2.54 GB (by Equation (1), $S \leq \frac{0.8s}{1/6.4 \text{ GB/s} + 1/6.3 \text{ GB/s}} = 2.54 \text{ GB}$), which is much larger than 1200 MB. This indicates that memory access patterns of DNN workloads exhibit the property of extremalization. Those outlier memory behaviors with high ΔT s and large memory block size ought to be the focus of attention. They are the major contributors in terms of reducing the device memory pressure of DNN training.

3.3 Swapping Behaviors between Host and Device Memory

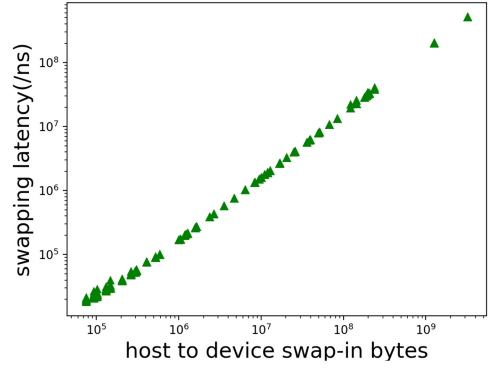
Our characterization shows that the bandwidth between host and device will affect the boundary swapping size for different access time intervals. In the DNN domain, the memory block size varies greatly for different input datasets and batch size. It is necessary to measure the data swapping latency between host and device with different swapping bytes. Obviously, we can see from Figure 6 that the data swapping latency T_{sw} between host and device memory is proportional to the number of swapping bytes s , which can be expressed with a linear model: $T(s) = B * s + \text{startup overhead}$. Besides, the DNN accelerator vendors usually provide similar APIs to manage the device memory and execution orders and swap data back and forth between the host CPUs and device accelerators (see Table 1).

From the case study of MLP training and the swapping behaviors, we highlight the following observations:

- **Patterns of Memory Behaviors:** The memory access patterns of DNN workloads exhibit the properties of *iterative*, *regularity*, and *extremalization*. Specifically, the memory



(a) Device to host data swapping latency.



(b) Host to device data swapping latency.

Fig. 6. Data swapping latency between host and device with different swapping size.

behaviors of DNN workloads illustrate obvious iterative characteristics (Figure 4(a)). Besides, the distributions of the access time intervals (ΔT s) of most memory behaviors are very stable. Especially, ΔT s of most memory access behaviors are negligible, which can be filtered out by the bandwidth between host memory and device memory (Figure 4). Most importantly, those outlier memory behaviors with high ΔT s and large memory block size ought to be the focus of attention (Figure 5). They are the major contributors in terms of reducing the memory pressure of DNN workloads.

- **Characteristics of Heterogeneous Platforms:** It is easy to model the swapping overhead between the host and device memory. The swapping latency is proportional to the number of swapping bytes (Figure 6). Furthermore, the accelerator vendors usually provide functional similar programming APIs to manage the device memory and execution orders (Table 1).

Based on the two observations, it is desired to provide a more general and versatile approach that takes the memory access patterns as input to automatically address the memory pressure issues of DNN workloads with small runtime overhead.

4 DESIGN AND IMPLEMENTATION

The distinctive memory access patterns of DNN workloads and the platform characteristics observed in Section 3 motivate the design of AppObMem. AppObMem tries to automatically reduce the memory pressure of DNN workloads with small runtime overhead.

4.1 AppObMem Workflow Overview

4.1.1 Design Overview. The goal of AppObMem is to automatically schedule the memory usage of DNNs workloads while maximizing the reduction of memory footprint and minimizing the runtime overhead. AppObMem is a runtime system running on the host-side that interfaces with the device accelerators to dynamically swap the memory blocks back and forth between the host and device memory. Figure 7 presents the workflow of AppObMem. As shown, AppObMem is completely transparent to end-users. It takes the bandwidth information between the host and device memory as input. Then, AppObMem traces the memory behavior sequences (including allocation, writing, reading, and free) of the DNN workloads by pinpointing the status of each device memory block via automatic instrumenting the allocators of the runtime system. Considering that DNN workloads illustrate obvious iterative characteristics, AppObMem only instruments the first several rounds of iterations to reduce the runtime overhead. After detecting the iterative memory

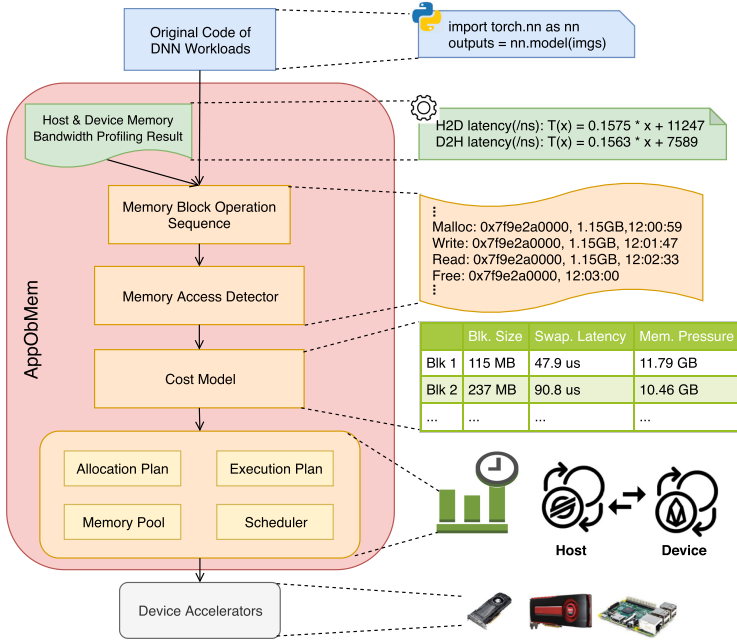


Fig. 7. Workflow overview of AppObMem.

access patterns of the DNN workloads, AppObMem builds an empirical cost model to decide which memory block to be swapped-out or swapped-in automatically. Based on the running results of the cost model, AppObMem generates an allocation plan and execution plan, which will be deployed and scheduled for the rest rounds of iterations of the DNN workloads.

4.1.2 Core Abstractions. In this section, we integrate the previous-mentioned modules into a general solution. Figure 8 shows the core data structures of AppObMem. To tackle the memory pressure issues of device accelerators, AppObMem makes some abstractions for the device accelerators from the perspective of memory. As illustrated, the class *DevAcc* provides some common APIs to manage the device memory and perform asynchronous execution, which unifies the typical runtime APIs provided by the accelerators vendors (Table 1). The device memory is split into chunks of memory blocks (class *DevMemBlock*), which are managed by a device memory pool (class *DevMemPool*). For Nvidia GPUs, the CNMeM library [44] can be an instance of class *DevMemPool*. AppObMem samples the *Write*, *Read* behaviors of *DevMemBlock* and *Malloc*, *Free* behaviors of *DevAcc*. The recorded meta-information includes device memory address, block size, and timestamp. These sampled memory access requests are used for the detection of iterative patterns. After detection, these iterative patterns are used to guide the scheduling of memory swapping. These core abstractions help the integration of AppObMem into existing deep learning frameworks with ease. This also helps the end-users run the DNN workloads without modifying the high-level python code provided by the deep learning frameworks. We will give a reference implementation and evaluation of AppObMem at the deep learning framework—Apache Singa [1] in Section 5.

4.2 Implementation Details

4.2.1 Memory Swapping Cost Model. AppObMem adopts the idea of memory swapping. Our characterization in Section 3 shows that access time intervals of most memory access behaviors

```

// Meta-info for memory scheduling.
struct SchedMetaInfo {
    void *dev_ptr, *cpu_ptr;
    size_t block_size = 0;
    vector<long> time_stamp;
    DevAsyncEvent in_event, out_event;
    DevAsyncStream in_stream, out_stream;
};

// Abstractions of device accelerators.
class DevAcc {
    DevAcc(DEV_ACC_TYPE, DevMemPool*,...);
    void Malloc(...);
    void Free(...);
    void Malloc(...);
    void MallocAsync(...);
    void AsyncExec(...);

private:
    // can be GPU, Ascend910, etc.
    DEV_ACC_TYPE type;
    DevMemPool *pool;
};

class DevMemBlock {
public:
    Block(void *ptr, size_t size, size_t offset, DevAcc *dev);
    void* Write();
    const void* Read();

    size_t block_size = 0;
    size_t offset = 0;
    void *ptr = nullptr;
    DevAcc *dev;
};

class DevMemPool {
    virtual void Malloc(void **ptr, const size_t size) = 0;
    virtual void Free(void *ptr) = 0;
};

class AppObMem {
    virtual void Detect() = 0;
    virtual void Schedule() = 0;

private:
    DevMemPool *pool;
    map<DevMemBlock*, SchedMetaInfo> meta_sched;
};

```

Fig. 8. Core abstractions of AppObMem.

are negligible. Those outlier memory behaviors with high access time intervals and large memory block size are the major contributors to reduce the memory pressure of DNN workloads. We formalize an empirical cost model to guide the selection of these outlier memory behaviors. Given some sequence of n memory access requests: $R_0, R_1, \dots, R_{n-1}$. A memory access request R_i happens at time T_i is marked as a tuple of memory access action A_i to memory block B_i , i.e., $R_i := \langle A_i, B_i, T_i \rangle$, where $A_i \in \{\text{malloc}, \text{free}, \text{write}, \text{read}\}$. Let S_i be the size of memory block B_i ; H_k be the highwater mark of memory payload over the first k memory access requests. Then we have

$$H_k = \sum_{i=0}^k \alpha_i * S_i, \alpha_i \in \{-1, 0, 1\},$$

where $\alpha_i = 0$ iff A_i is *read* or *write*; $\alpha_i = 1$ iff A_i is *malloc*; $\alpha_i = -1$ iff A_i is *free*. H_k reflects current memory pressure of device accelerator. The higher the value of H_k , the greater the current memory pressure.

Our statistics in Section 3.3 show that the swapping latency between the host and device memory is easy to model. We use an empirical linear model to characterize the swapping latency with different swapping bytes. Let $T_{sw-in}(S_i)$ ($T_{sw-out}(S_i)$) be the swap-in (swap-out) overhead of current memory block B_i with block size S_i ; then we have

$$T_{sw-in}(S_i) = \beta_{h2d} * S_i + O_{in},$$

$$T_{sw-out}(S_i) = \beta_{d2h} * S_i + O_{out},$$

where β is the swapping bandwidth and O is the starting up overhead. Note that, for a given heterogeneous computing system, β and O can be treated as runtime constants, which can be pre-obtained by offline profiling. To determine β and O , we can sample a group of swapping latency t with different swapping bytes s at the given heterogeneous computing system, marked as $(t_0, s_0), (t_1, s_1), \dots, (t_n, s_n)$. Let us assume the fitting error is ϵ . Then the linear model should satisfy

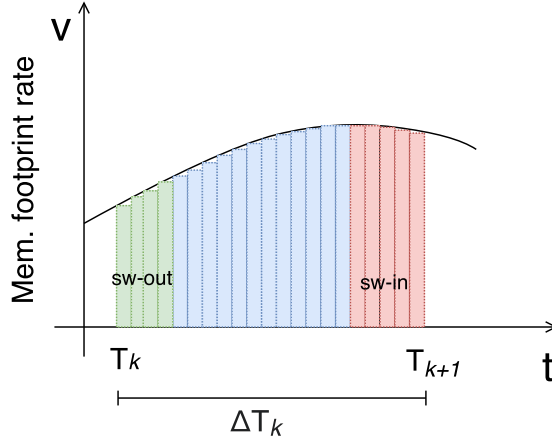


Fig. 9. Illustration of memory saving gain, whose design refers to the velocity-time model in physics. Note that the blue part means the valid memory saving gain of swapping current device memory block.

the condition that the sum of the square errors of all the observed values t_i and the corresponding regression estimates $\hat{t}_i$ is the minimum. So we have the following:

$$\arg \min_{\beta, O} \sum_{i=0}^n \epsilon_i^2 = \arg \min_{\beta, O} \sum_{i=0}^n (t_i - \hat{t}_i)^2, \quad (2)$$

$$= \arg \min_{\beta, O} \sum_{i=0}^n (t_i - \beta * s_i - O)^2. \quad (3)$$

Equation (3) can be solved by the least squares method [63]. We define the memory saving gain of swapping out the memory block B_k as G_k :

$$G_k = \frac{S_k}{H_k} * T_{valid}^{(k)},$$

$$T_{valid}^{(k)} = \Delta T_k - T_{sw-in}(S_k) - T_{sw-out}(S_k).$$

Here H_k denotes the highwater mark of memory payload over the first k memory access requests, which represents the device memory pressure. ΔT_k is the access time interval between two consecutive memory accesses to the device memory block B_k . $T_{valid}^{(k)}$ denotes the valid elapsed time that memory block B_k is not in the device memory. The negative values of $T_{valid}^{(k)}$ usually mean the inevitability of runtime overhead. We know that DNN workloads are usually very time-consuming. For example, it usually takes weeks or months to train a DNN. Even a moderate runtime overhead would lead to a large time cost, which is unacceptable in the production environment. It is necessary to consider the swapping overhead. Our previous characterization in Section 3.2 shows that those outlier memory behaviors with high access time intervals and large memory block size ought to be the focus of attention. Therefore, we take the size (S_k) of memory block B_k into consideration.

G_k is an empirical metric. As shown in Figure 9, its design refers to the velocity-time model in physics. In physics, the physical meaning of the integral of velocity along the time dimension is displacement. Similarly, $\frac{S_k}{H_k}$ approximately characterizes the current device memory footprint rate. G_k is the product of $\frac{S_k}{H_k}$ and $T_{valid}^{(k)}$. The larger the product of the two indicates that, under the premise of the current severe device memory pressure (higher $\frac{S_k}{H_k}$), swapping the device memory

blocks with longer access time interval (higher $T_{valid}^{(k)}$) can bring more effective memory saving benefits. AppObMem uses G_k to guide the selection of memory behaviors. A candidate memory behavior $\langle A_k, B_k, T_k \rangle$ with larger G_k is preferred for the swapping of memory block B_k .

ALGORITHM 1: Memory Swapping Scheduling

```

// vector<R>: a sequence of memory access requests
Function Main(vector<R>):
  for Each <A, B> in vector<R>; do
    // Instrument the memory pool allocator, for each Allocation, Release, Read, Write
    Instrument and record meta_info for each memory block B, including: (access action A, device address pointer P of B,
    block size S of B, timestamp T);
  // The detection of iterative memory access patterns can be reduced to the longest common substring
  problem.
  // It can be solved by dynamic programming algorithm.
  Detect iteration repetition patterns of vector<R> with meta_info;
  Schedule(vector<R>, meta_info);
  return;

Function Schedule(vector<R>, meta_info):
  Calculate memory-saving gain G for the memory requests in vector<R>;
  Generate scheduling table meta_sched with meta_info;
  Sort G and determine candidate swapping memory blocks;
  Update meta_sched; // Record async events & streams
  Construct memory pool with meta_sched and meta_info;
  PerformSwapping(pool, meta_sched);
  return;

Function PerformSwapping(pool, meta_sched):
  Get swapping flags from meta_sched;
  if swapping flag is SwappingIn; then
    Malloc device memory for current block from pool;
    Create async stream and MemcpyAsync(Host2Device);
    Update current block's meta info and record this event;
  if swapping flag is SwappingOut; then
    Create async stream and MemcpyAsync(Device2Host);
    Update current block's meta info and record this event;
  Get sync flags from meta_sched;
  if sync flag is SyncSwappingIn; then
    Synchronize the asynchronous event for corresponding swapping-out stream of current memory block;
    Update current block's address pointer into new device address;
    Update meta info for current block;
  if sync flag is SyncSwappingOut; then
    Synchronize the asynchronous event for corresponding swapping-in stream of current memory block;
    Update current block's address pointer into nullptr;
    Free current memory block from pool;
    Update meta info for current block;
  return;
  
```

4.2.2 Memory Swapping Scheduler. The cost model defined in Section 4.2.1 determines *which memory blocks can be selected for swapping*. We know that if the swapping-in event happens too late, then this will introduce much runtime overhead. However, if prefetching data happens too early, then the memory-saving gains of AppObMem will be reduced. Therefore, it is important to decide *when to perform the swapping-out and swapping-in actions*.

The implementation details of the memory swapping scheduler are presented in Algorithm 1. First, AppObMem samples and traces the allocation, release, reading, and writing behaviors of the allocator and records the meta-information for each memory request so that to detect the iteration patterns of the DNN workloads. The instrumented meta-information includes the access

actions (can be malloc, free, write, and read), the device memory address, size of the memory block, and the corresponding timestamp. The detection of iterative memory access patterns can be reduced to the *longest common substring problem*, which can be solved by dynamic programming algorithm [67]. AppObMem uses the meta-information and iteration patterns to schedule the memory swapping.

To choose the swapping memory blocks from the candidate memory behaviors, AppObMem calculates the memory-saving gains G_k for all candidates. These candidates are then sorted by their memory-saving gains. AppObMem generates a scheduling table for these selected swapping blocks. The scheduling table is a map of the selected swapping blocks and their swapping events. The key is the block index, the value is the meta-information related with the swapping events. For each swapping block, the following meta-information is recorded: original device memory address, host CPU memory address, size of the block, asynchronous swapping-in/-out event, and asynchronous swapping-in/-out stream. Besides, the flags for the swapping and synchronization events of the swapping-in and swapping-out actions are also necessary. These flags denote when to perform the swapping events and when to perform the synchronization events. AppObMem uses a memory pool to manage the device memory. Then, AppObMem will construct the memory pool and records the meta-information of these selected swapping blocks.

To maximize the swapping and synchronization overlap ratio, AppObMem employs separate streams to swap the selected memory blocks in/out of external memory asynchronously by calling the async execution APIs provided by the accelerators vendors, e.g., *cudaStream_t* for Nvidia GPUs. These asynchronous swapping streams are responsible for data swapping, allocation, and release. To ensure correctness, the scheduler works in such a way that only after the previous swapping-in/-out event is finished does the memory block starts to be swapped-in/-out. In particular, AppObMem schedules the swapping-in events in line with the next access time of current memory block. To ensure that the second access is not delayed by the swapping events, the memory block needs to be swapped-in in advance. For those swapping-out events, they are also scheduled in chronological order. A memory block can be swapped-out immediately after the access is completed.

When perform swapping, AppObMem queries the swapping flags and the synchronization flags from the scheduling table. If the swapping flag is *SwappingIn*, then AppObMem will allocate a device memory block from the memory pool and asynchronously copy data from the host CPUs to the device accelerators. Meanwhile, AppObMem has to update the meta-information of this block and record this asynchronous event for the overlap of swapping and synchronization. The case is similar for swapping-out. If the synchronization flag is *SyncSwappingIn*, then AppObMem will perform the following actions: synchronizing the asynchronous event for the swapping-out stream of current memory block and updating current block's device memory address to the new address. Meanwhile, AppObMem has to update the meta-information of this block. Similarly, for *SyncSwappingOut*, AppObMem has to perform synchronizing, updating current block's device memory address into *nullptr* and deallocating this block. Besides, current block's meta-information also has to be updated. In this case, swapping and synchronization can be completely hidden under computation.

5 EVALUATION

Our evaluation is conducted on the Nvidia Titan X Pascal GPU with 12-GB GDDR5X device memory and 3840 CUDA cores, and the newer Nvidia Geforce RTX 2060 (Turing Arch) GPU with 6GB GDDR6 device memory and 1920 CUDA cores. The host machine is an Intel Xeon E5-2630 v3 CPU @2.40 GHz with 16 cores/32 threads and 20 MB of L3 cache per socket. The host CPU is equipped with 32 GB DDR4 RAM. The operating system is Ubuntu 16.04.10 LTS with cuDNN-v7.6.5 and

Table 4. Benchmarked DNNs

Application Scenarios	Computer Vision		Natural Language Processing		Machine Learning
DNN Models	Linear Topology	Non-linear Topology	LSTM-based	Transformer-based	
	VGG [59]	GoogLeNet [61], ResNet [22] DenseNet [25], Inception [60] NASNet [71], Auto-Encoder [35] DCGAN [53]	Char-RNN [31] RNN [58] Bi-RNN [57]	BERT [18]	K-Nearest-Neighbor [14] GBDT [52] KMeans [40] Logistic Regression [4]
Datasets	CIFAR-100 (32 × 32) ImageNet (224 × 224) COCO (300 × 300)		TC Wikipedia [2]	SQuAD [54]	Synthetic

CUDA-v10.2. We measure the swapping bandwidth and starting up overhead of the heterogeneous platform in Section 4.2.1 through offline profiling. For Titan X Pascal GPU, β_{h2d} is 0.1575, O_{in} is 11247, β_{d2h} is 0.1563, and O_{out} is 7589. We finished a reference implementation of AppObMem on the GPU device and integrate it into Apache Singa. Singa is an Apache top-level project, focusing on the training of DNNs. Singa internally optimizes the training speed and memory footprint of DNN workloads [1]. We choose it as our baselines.¹

5.1 Benchmarks

As for the benchmarks, we sift out some typical DNNs from computer vision and natural language processing. To verify the generality of AppObMem, we also choose some machine learning models. The details are presented at Table 4. VGG are linear DNNs, whose data are usually propagated in a sequential order during both forward and backward propagation. Compared with linear DNNs, GoogLeNet, ResNet, DenseNet, and NASNet are non-linear ones. There are many forks and joins in their layer structures. As for the dataset, we choose some light-weight ones, like CIFAR-100 (with input size 32 × 32), and the heavy-weight ones, like ImageNet (224 × 224) and COCO (300 × 300). For LSTM-based RNNs, we variable the hidden size and sequence length of the input for its LSMT layers [58]. For machine learning models, we use the randomly generated synthetic datasets.

5.2 Experimental Results

To show the effectiveness of AppObMem, we first show its overall performance in terms of memory footprint reduction and runtime overhead. We also compare AppObMem with some open sourced SOTA schemes, such as SuperNeurons [65] and TF-LMS [34].

5.2.1 Reduction on Memory Footprint. Since AppObMem is integrated into Apache Singa, we choose Singa disabling the memory scheduling optimization as the baselines. The Nvidia GPU command-line tool (*nvidia-smi*) fails to capture the realtime device memory footprint. So we use the *cudaMemGetInfo* from CUDA SDK to monitor the fine-grained realtime device memory usage of GPU. The memory footprint of AppObMem usually follows an iterative manner. For the sake of fairness, we choose the peak memory footprint as the memory pressure of current DNN workload.

We have benchmarked AppObMem with different input datasets, batch size, and model layer structures. Figure 10(a), (b), (c) presents the normalized memory usage among AppObMem and the baselines for the linear DNNs (VGG), and the non-linear DNNs, like GoogLeNet, ResNet, DenseNet, and Char-RNN. We observe that, compared with the baselines, AppObMem obtains about 40–60% memory-saving gains for all the training workloads. Besides, AppObMem works well on all input datasets with different batch size. This mainly attributes to the fact that AppObMem captures the unique memory access patterns of these DNN workloads. Our experimental results also uncover that AppObMem is not an over-fitting for a certain dataset, batch size, or a certain layer structure of DNNs. AppObMem is not sensitive to these involving factors.

¹The code of this work is publicly available at this GitHub repository: <https://github.com/Phaeton-lang/baselines>.

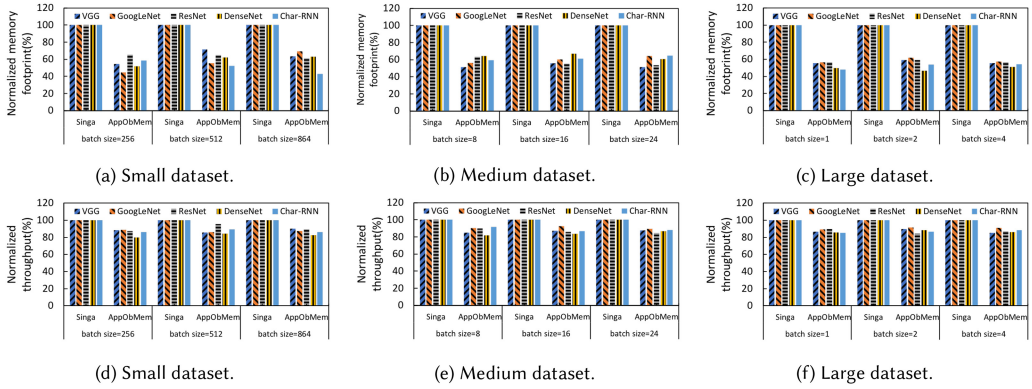


Fig. 10. Normalized memory footprint and training throughput comparison of AppObMem and baselines. Small dataset, CNNs: CIFAR-100 (32×32), Char-RNN (voc size: 67, hidden size: 128, seq len: 64). Medium dataset, CNNs: ImageNet (224×224), Char-RNN (voc size: 67, hidden size: 1024, seq len: 512). Large dataset, CNNs: COCO (300×300), Char-RNN (voc size: 67, hidden size: 2048, seq len: 1024).

Table 5. Runtime Overhead Breakdown of AppObMem for DNN Training under the ImageNet Dataset with Batch Size 16

	VGG	GoogLeNet	ResNet	DenseNet	Char-RNN
# Iteration Patterns	2845	1002	3756	23434	9234
Instrument Meta-Info (µms)	121.09 (14.44%)	7.990 (14.60%)	23.487 (1.86%)	233.11 (7.25%)	0.047 (0.003%)
Pattern Detection (µms)	716.10 (85.38%)	46.70 (85.33%)	1241.5 (98.11%)	2959.5 (92.04%)	1764.22 (99.99%)
Block Selection (µms)	0.7890 (0.09%)	0.018 (0.03%)	0.3842 (0.03%)	12.050 (0.37%)	0.0628 (0.002%)
Generate Scheduling (µms)	0.7112 (0.08%)	0.020 (0.04%)	0.1047 (0.01%)	10.865 (0.34%)	0.039 (0.003%)
Total (µms)	838.69 (100%)	54.73 (100%)	1265.5 (100%)	3215.5 (100%)	1764.36 (100%)

5.2.2 Runtime Overhead Analysis. To measure the runtime overhead, we use the end-to-end training throughput as the metrics. Figure 10(d), (e), (f) shows the training throughput comparison of AppObMem to baselines. As illustrated, AppObMem introduces tolerable runtime overhead for these training workloads. Compared with the non-swapping versions, AppObMem sacrifices only about 10% of runtime performance on average. For some use cases, the runtime overhead of AppObMem is even negligible. For example, for ResNet training with CIFAR-100 and batch size of 512, its training throughput is about 97% of the baselines. Meanwhile, its memory footprint is only about 60% of the baselines. This is very useful in real-world production environments, such as data centers. That means we can use much less memory to train the DNNs without significantly sacrificing runtime performance. In the production environments, considering the memory and throughput requirements of DNN workloads, we can use the Pareto optimality criterion to do tradeoff between memory footprint and throughput [41].

We dump some statistical information of AppObMem, including the number of iteration patterns, the time consumption of tracing meta-information for the memory requests, pattern detection time and the time cost of candidate memory block selection and generation of scheduling table. Table 5 presents the detailed runtime overhead breakdown of AppObMem for different DNN workloads. We observe that the runtime overhead of AppObMem ranges from tens of microseconds to a few seconds. Compared with the training cost for a commercialized DNN (usually days or weeks), the runtime overhead of AppObMem is negligible. Specifically, the overhead of different DNN workloads varies greatly. Among these DNNs, DenseNet is the most time-consuming in

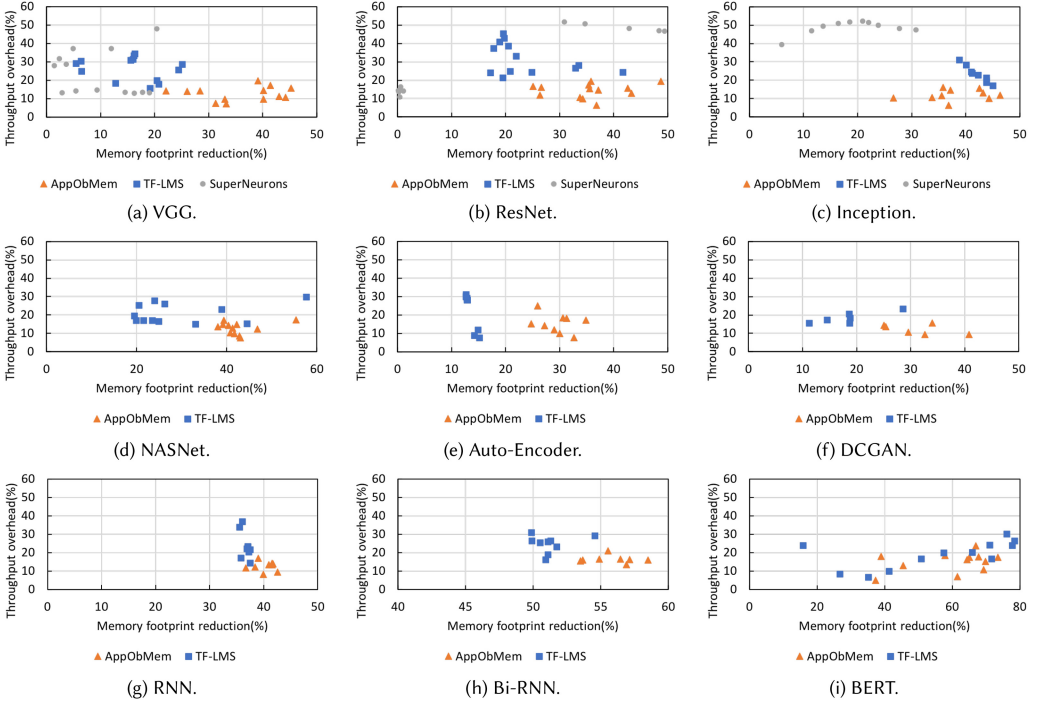


Fig. 11. Memory footprint reduction and throughput overhead comparison of AppObMem and other SOTA open source solutions on CV and NLP models. The lower right of the data points in each figure means the better the effects.

terms of memory scheduling overhead. This is mainly due to its large amounts of temporary results and frequent memory access during the training of DenseNet. For DenseNet, each layer obtains additional inputs from all preceding layers and passes on its own feature maps to all subsequent layers, trending to generate many temporary results.

5.2.3 Cross-solution Comparisons. We also evaluated AppObMem with some SOTA open sourced solutions, e.g., SuperNeurons [65] and TF-LMS [34]. Due to the great difference between different memory scheduling systems, it is difficult to balance the effect of memory scheduling with the absolute value of memory footprint and training throughput. For the sake of fairness, we compare their ratio of memory footprint reduction and throughput overhead (Equation (4)),

$$\begin{aligned}
 R_{\text{memory footprint reduction}} &= \left(1 - \frac{M_{\text{after optimization}}}{M_{\text{before optimization}}}\right) * 100\%, \\
 R_{\text{throughput overhead}} &= \left(1 - \frac{F_{\text{after optimization}}}{F_{\text{before optimization}}}\right) * 100\%.
 \end{aligned}
 \tag{4}$$

As shown in Figure 11, in general, compared with SuperNeurons and TF-LMS, AppObMem is comparable in terms of device memory savings and runtime overhead. For RNNs and BERT in Figure 11(g), (h), (i), in most cases, AppObMem achieves better device memory savings, and its runtime overhead is lower than TF-LMS. Since SuperNeurons supports fewer models in terms of test cases, we only evaluated its performance on the linear VGG and the non-linear ResNet and Inception. We observe from Figure 11(a), (b), (c) that, compared with other schemes, SuperNeurons

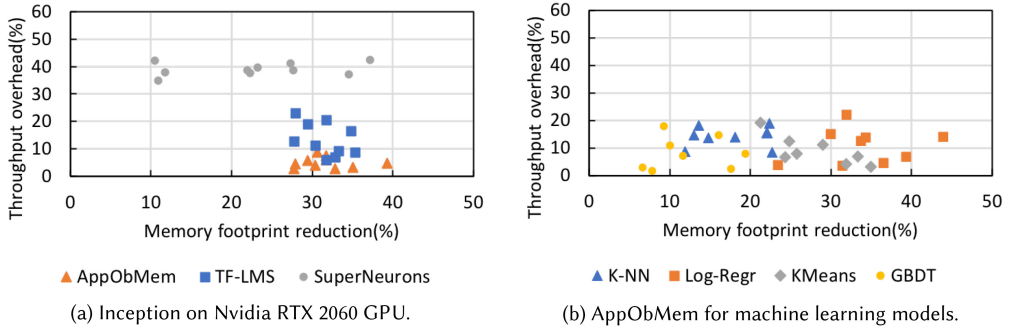


Fig. 12. Memory footprint reduction and throughput overhead comparison of AppObMem and other SOTA open source solutions on Nvidia RTX 2060 (Turing) GPU for Inception and machine learning models. The lower right of the data points in each figure means the better the effects.

is badly behaved. This is not surprising, as SuperNeurons is implemented by handwritten C++ and cuDNN, which is not comparable to AppObMem (Apache Singa) and TF-LMS (TensorFlow) that are based on industrial solutions.

We also evaluated on NVIDIA GeForce RTX 2060 GPU. RTX 2060 is powered by the Nvidia Turing architecture. It is equipped with 1920 CUDA cores and 6 GB GDDR6 device memory. Its memory bandwidth is 336 GB/s. As presented in Figure 12(a), compared with SuperNeurons and TF-LMS, AppObMem still works better in terms of device memory savings and runtime overhead.

Machine learning models cannot be easily expressed by conventional computational graphs. Existing memory scheduling schemes mainly rely on the application information, such as the topology of computational graphs; they cannot handle the machine learning models well and even introduce additional runtime overhead. We only report the performance of machine learning models on AppObMem. As shown in Figure 12(b), AppObMem can still handle these machine learning models well.

However, compared with CNNs and RNNs in Figure 11, AppObMem can only save about 17.8%, 33.8%, 28.2%, and 12.3% of device memory for k-NN, Log-Regr, KMeans, and GBDT, respectively. The main reason is that compared with the regular convolution, matrix multiplication and other operations in DNN model, the machine learning models contain a lot of sorting, reduction, and other operations, which are not so regular in terms of memory access patterns. For example, for the GBDT model in Figure 13(d), there are not so many outliers with long ΔT s and large memory blocks size, so the device memory savings it obtains from AppObMem are not obvious. For this kind of model, their further reduction of device memory pressure can be considered from the allocation strategy of memory pool [56].

5.2.4 Potential Application Scenarios of AppObMem in the Multiple-Users-Contention Use Case.

To extend AppObMem into the multiple-users-contention use case in the cloud environment, we quantitatively analyzed the potential application scenarios of AppObMem using a single server with multiple users as an example.

Assume that a server is equipped with only one GPU card and its device memory capacity is M . There are multiple users competing for this GPU server at the same time. Suppose that user A and user B need to train their own DNN models in this GPU server. Without the use of AppObMem, the device memory required by the DNN model of user A and user B is M_A and M_B , respectively. Assume that the time cost for user A to train his DNN model is C_A , and the time cost for user B to train his DNN model is C_B . If $M_A + M_B > M$ happens, then the GPU resource contention would

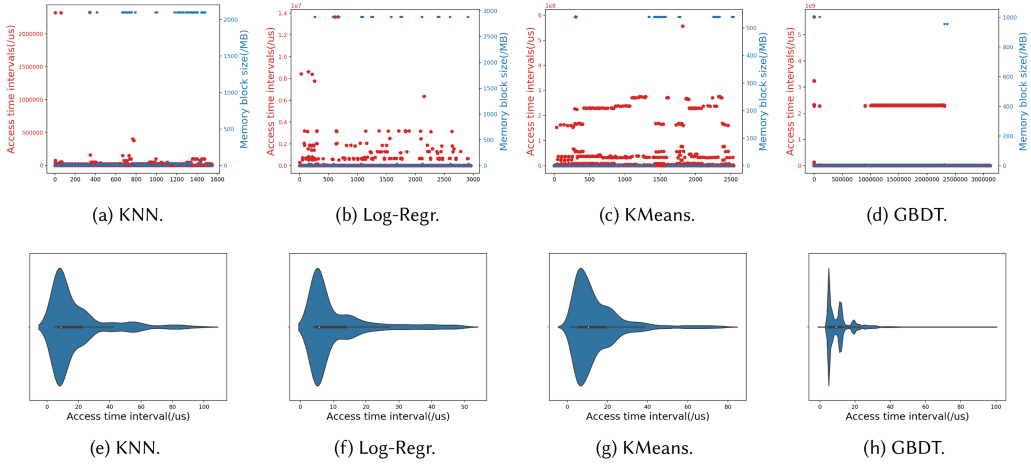


Fig. 13. AppObMem memory behaviors pattern analysis for machine learning models.

occur. In this case, the time cost of completing the training tasks of users A and B on this GPU server is $C_A + C_B$. Then we deploy AppObMem onto this GPU server. Assume that AppObMem brings a performance loss of l_A to user A 's model training and a memory-saving gain of g_A . Similarly, the performance penalty for user B 's model training is l_B and the memory-saving benefit is g_B . AppObMem can be applied in this scenario where user A and user B can train their own models at the same time. In this use case, the time cost required for user A and user B to complete their own training tasks is $\max(\frac{C_A}{1-l_A}, \frac{C_B}{1-l_B})$. Assume that user A executes the training task first. The restrictions are as follows:

$$\begin{cases} M_A \leq M \\ M_B \leq M \\ M_A + M_B > M \\ M_B \leq M - M_A * (1 - g_A) \end{cases} \quad (5)$$

Then the solution objective is $C_A + C_B \geq \max(\frac{C_A}{1-l_A}, \frac{C_B}{1-l_B})$. According to the performance data in our experimental section, we set $g_A = 60\%$, $l_A = 10\%$, and $l_B = 10\%$. Assume that the device memory capacity of the GPU server is 12 GB. Therefore, the problem is formulated to solve $C_A + C_B \geq \max(\frac{C_A}{1-10\%}, \frac{C_B}{1-10\%})$ such that the following constraints are satisfied:

$$\begin{cases} M_A \leq 12 \\ M_B \leq 12 \\ M_A + M_B > 12 \\ M_B \leq 12 - M_A * (1 - 60\%) \end{cases} \quad (6)$$

Under the above constraints, there are still many DNN models satisfying these constraints. It is assumed that the training model of user A is more time-consuming, i.e., $C_A \geq C_B$, $\max(\frac{C_A}{0.9}, \frac{C_B}{0.9}) = \frac{C_A}{0.9}$. Therefore, the solution target is $C_A + C_B \geq \frac{C_A}{0.9}$, and then we have $\frac{C_A}{9} \leq C_B \leq C_A$. That means, as long as the model training time cost of user B does not exceed the model training time cost of user A , and the model meets the constraint condition 6 on the device memory requirement, user A and user B can simultaneously execute the training task on the GPU server with the help of AppObMem. Figure 14 presents the feasible solution region of constraint conditions 6. The blue part in Figure 14 represents the training space that user A and B can train their DNN models without using AppObMem. Because of AppObMem, we have expanded this area to the red part.

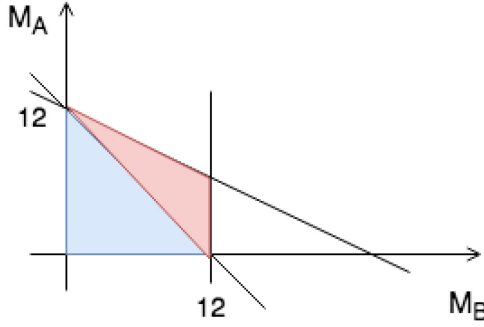


Fig. 14. Feasible solution region of constraint conditions 6.

The area of the red part accounts for 60% of the area of the blue part. Therefore, we conclude that AppObMem can theoretically increase the maximum usage of this GPU server by 60%.

Based on the above experiments, we conclude that memory-efficient training optimization from the perspective of raw memory access patterns is feasible. AppObMem can effectively handle various linear and nonlinear DNN workloads, including CNNs, LSTM-based RNNs, transformer-based BERT, and even machine learning models. Compared with other SOTA open source solutions, AppObMem is also competitive.

6 LIMITATIONS OF APPOBMEM

As an infrastructure between the high-level DNN workloads and the underlying accelerators, AppObMem works well for typical linear and non-linear DNN workloads. However, AppObMem has the following limitations (we leave the exploration of these aspects for future work):

- The main runtime overhead of AppObMem is the online tracing of memory behaviors and pattern detection for the previous rounds of iterations (Table 5). These overheads can be further squeezed by offline profiling;
- AppObMem currently only supports memory scheduling for training workloads on a single DNN accelerator. As for the inference workloads with only forward propagation, the ΔT s of memory blocks are closely related to the layer structures and dependency of execution orders. A DNN with long forks may tend to generate long ΔT . One potential solution is to relax the cost model in Section 4.2.1 and even adopt an aggressive swapping strategy that only swaps partial data of some memory blocks;
- As for the multiple device accelerators support, the typical methods are data parallelism [32] and model parallelism [16]. For data parallelism, each device accelerator maintains a complete copy of the given DNN and processes disjoint parts of the input data. In this case, we may deploy AppObMem on each device accelerator to reduce the single node's memory pressure. Model parallelism divides the given DNN into several subnets and assigns the disjoint subnets to a single device accelerator. For this case, the communication and synchronization overhead between different accelerators are the bottleneck, which is beyond the scope of AppObMem.
- Memory swapping-based schemes could have implications on security or privacy aspect due to exposing intermediate activations to the host. A potential solution for this problem is to introduce a lightweight checking mechanism. That is, an extra parity bit can be asynchronously generated before the current memory block is swapped out. Then the parity operation is inserted before the swapping-in actions to ensure that the swapped

intermediate activations are not modified. This checking mechanism may introduce additional performance overhead.

- In the cloud environment with multi-user preemption in the multi-tenant clusters, there may be changes in the application behavior specific to the memory patterns. In this case, how to support elasticity is important, i.e., the ability to adaptively amend the scheduling strategies during DNN training. Elasticity in the multi-tenant cluster management can be implemented by achieving various scheduling objectives (e.g., memory occupation, job throughput, and resource efficiency) according to the cluster load variations [39, 68].

7 CONCLUSION

This article aims to tackle the memory challenge of DNN workloads by proposing AppObMem. The distinctive memory behaviors of DNN workloads and the unified characteristics of heterogeneous platforms motivate the design of AppObMem. AppObMem automatically traces the memory behaviors of DNN workloads and schedules the memory swapping to reduce the memory pressure of the device accelerators without the perception of high-level application features. Experimental results show that AppObMem obtains about 40–60% memory savings with acceptable runtime overhead for the training of typical linear and non-linear DNNs. Compared with other SOTA open sourced schemes, AppObMem is also competitive.

REFERENCES

- [1] Apache Singa Team. 2020. Singa: A Distributed Deep Learning Library. Retrieved September 20, 2020 from <http://singa.apache.org/>.
- [2] Javier Artiles and Satoshi Sekine. 2008. Tagged and Cleaned Wikipedia (TC Wikipedia) and Its Ngram. Retrieved September 20, 2020 from <https://nlp.cs.nyu.edu/wikipedia-data/>.
- [3] Olivier Beaumont, Lionel Eyraud-Dubois, and Alena Shilova. 2019. Optimal GPU-CPU Offloading Strategies for Deep Neural Network Training. Retrieved from <https://hal.inria.fr/hal-02316266>.
- [4] Giuseppe Bonaccorso. 2017. *Machine Learning Algorithms: A Reference Guide to Popular Algorithms for Data Science and Machine Learning*. Packt Publishing.
- [5] Cambricon. 2020. Cambricon BANG C Developer Guide. Retrieved September 20, 2020 from http://www.cambricon.com/docs/bangc/developer_guide.html/.
- [6] Hanting Chen, Yunhe Wang, Chunjing Xu, Boxin Shi, Chao Xu, Qi Tian, and Chang Xu. 2020. AdderNet: Do we really need multiplications in deep learning? In *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition Conference (CVPR '20)*.
- [7] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. 2015. MXNet: A flexible and efficient machine learning library for heterogeneous distributed systems. arXiv:1512.01274. Retrieved from <http://arxiv.org/abs/1512.01274>.
- [8] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. CoRR abs/1604.06174 (2016). arXiv:1604.06174 <http://arxiv.org/abs/1604.06174>.
- [9] Xiaoming Chen, Danny Z. Chen, and Xiaobo Sharon Hu. 2018. moDNN: Memory optimal DNN training on GPUs. In *Proceedings of the Design, Automation & Test in Europe Conference & Exhibition (DATE'18)*. IEEE, 13–18.
- [10] Yunji Chen, Tao Luo, Shaoli Liu, Shijin Zhang, Liqiang He, Jia Wang, Ling Li, Tianshi Chen, Zhiwei Xu, Ninghui Sun, and Olivier Temam. 2014. DaDianNao: A machine-learning supercomputer. In *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-47)*. IEEE Computer Society, Los Alamitos, CA, 609–622. <https://doi.org/10.1109/MICRO.2014.58>
- [11] Yu Cheng, Duo Wang, Pan Zhou, and Tao Zhang. 2017. A survey of model compression and acceleration for deep neural networks. Retrieved from <http://dblp.uni-trier.de/db/journals/corr/corr1710.html#abs-1710-09282>.
- [12] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient primitives for deep learning. Retrieved from <http://dblp.uni-trier.de/db/journals/corr/corr1410.html#ChetlurWVCTCS14>.
- [13] Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. 2015. BinaryConnect: Training deep neural networks with binary weights during propagations. In *Advances in Neural Information Processing Systems 28*, C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett (Eds.). Curran Associates, Inc., 3123–3131.
- [14] Thomas Cover and Peter Hart. 1967. Nearest neighbor pattern classification. *IEEE Trans. Inf. Theory* 13, 1 (1967), 21–27.

- [15] Henggang Cui, Hao Zhang, Gregory R. Ganger, Phillip B. Gibbons, and Eric P. Xing. 2016. GeePS: Scalable deep learning on distributed GPUs with a GPU-specialized parameter server. In *Proceedings of the 11th European Conference on Computer Systems (EuroSys'16)*. Association for Computing Machinery, New York, NY. <https://doi.org/10.1145/2901318.2901323>
- [16] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc'aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, Quoc V. Le, and Andrew Y. Ng. 2012. Large scale distributed deep networks. In *Advances in Neural Information Processing Systems 25*, F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger (Eds.). Curran Associates, Inc., 1223–1231.
- [17] Misha Denil, Babak Shakibi, Laurent Dinh, Marc' Aurelio Ranzato, and Nando de Freitas. 2013. Predicting parameters in deep learning. In *Advances in Neural Information Processing Systems 26*, C. J. C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Q. Weinberger (Eds.). Curran Associates, Inc., 2148–2156.
- [18] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805. Retrieved from <http://arxiv.org/abs/1810.04805>.
- [19] Amir Gholami, Zhewei Yao, Sehoon Kim, Michael W. Mahoney, and Kurt Keutzer. 2021. AI and Memory Wall. Retrieved from <https://medium.com/riselab/ai-and-memory-wall-2cb4265cb0b8>.
- [20] Adrunas Gruslys, Remi Munos, Ivo Danihelka, Marc Lanctot, and Alex Graves. 2016. Memory-efficient backpropagation through time. In *Advances in Neural Information Processing Systems*, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett (Eds.), Vol. 29. Curran Associates, Inc., 4125–4133.
- [21] Song Han, Huizi Mao, and William J. Dally. 2016. Deep compression: Compressing deep neural network with pruning, trained quantization and Huffman coding. In *Proceedings of the 4th International Conference on Learning Representations (ICLR'16)*, Yoshua Bengio and Yann LeCun (Eds.).
- [22] K. He, X. Zhang, S. Ren, and J. Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR'16)*. 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [23] K. Hornik, M. Stinchcombe, and H. White. 1989. Multilayer feedforward networks are universal approximators. *Neural Netw.* 2, 5 (July 1989), 359–366. [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)
- [24] Chien-Chin Huang, Gu Jin, and Jinyang Li. 2020. SwapAdvisor: Pushing deep learning beyond the GPU memory limit via smart swapping. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20)*. Association for Computing Machinery, New York, NY, 1341–1355. <https://doi.org/10.1145/3373376.3378530>
- [25] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger. 2017. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR'17)*. 2261–2269. <https://doi.org/10.1109/CVPR.2017.243>
- [26] Huawei. 2020. Atlas 300T Training Card (Model: 9000). Retrieved September 20, 2020 from <https://e.huawei.com/en/products/cloud-computing-dc/atlas/atlas-300t-training-9000>.
- [27] Huawei. 2020. Huawei Ascend Programming Guide. Retrieved September 20, 2020, from <https://www.huaweicloud.com/intl/en-us/ascend/doc/>.
- [28] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2017. Quantized neural networks: Training neural networks with low precision weights and activations. *J. Mach. Learn. Res.* 18, 1 (January 2017), 6869–6898.
- [29] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: Optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP'19)*. Association for Computing Machinery, New York, NY, 47–62. <https://doi.org/10.1145/3341301.3359630>
- [30] Norman P. Jouppi, Cliff Young, and Patil et al. 2017. In-datacenter performance analysis of a tensor processing unit. *SIGARCH Comput. Archit. News* 45, 2 (June 2017), 1–12. <https://doi.org/10.1145/3140659.3080246>
- [31] Yoon Kim, Yacine Jernite, David Sontag, and Alexander M. Rush. 2016. Character-aware neural language models. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI'16)*. AAAI Press, 2741–2749.
- [32] Alex Krizhevsky. 2014. One weird trick for parallelizing convolutional neural networks. arXiv:1404.5997. Retrieved from <http://arxiv.org/abs/1404.5997>.
- [33] Quoc V. Le, Jiquan Ngiam, Adam Coates, Abhik Lahiri, Bobby Prochnow, and Andrew Y. Ng. 2011. On optimization methods for deep learning. In *Proceedings of the 28th International Conference on International Conference on Machine Learning (ICML'11)*. Omnipress, Madison, WI, 265–272.
- [34] Tung D. Le, Haruki Imai, Yasushi Negishi, and Kiyokuni Kawachiya. 2019. Automatic GPU memory management for large neural models in TensorFlow. In *Proceedings of the ACM SIGPLAN International Symposium on Memory Management (ISMM'19)*. Association for Computing Machinery, New York, NY, 1–13. <https://doi.org/10.1145/3315573.3329984>

- [35] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. In *Proceedings of the IEEE*, Vol. 86. 2278–2324.
- [36] Guangli Li, Zhen Jia, Xiaobing Feng, and Yida Wang. 2021. LoWino: Towards efficient low-precision winograd convolutions on modern CPUs. In *Proceedings of the 50th International Conference on Parallel Processing*. 1–11.
- [37] Guangli Li, Xiu Ma, Xueying Wang, Lei Liu, Jingling Xue, and Xiaobing Feng. 2020. Fusion-catalyzed pruning for optimizing deep learning on intelligent edge devices. *IEEE Trans. Comput.-Aid. Des. Integr. Circ. Syst.* 39, 11 (2020), 3614–3626.
- [38] Guangli Li, Xiu Ma, Xueying Wang, Hengshan Yue, Jiansong Li, Lei Liu, Xiaobing Feng, and Jingling Xue. 2022. Optimizing deep neural networks on intelligent edge accelerators via flexible-rate filter pruning. *J. Syst. Arch.* 124 (2022), 102431.
- [39] Zihan Liu, Jingwen Leng, Zhihui Zhang, Quan Chen, Chao Li, and Minyi Guo. 2022. VELTAIR: Towards high-performance multi-tenant deep learning services via adaptive compilation and scheduling. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'22)*. Association for Computing Machinery, New York, NY, 388–401. <https://doi.org/10.1145/3503222.3507752>
- [40] J. B. MacQueen. 1967. Some methods for classification and analysis of MultiVariate observations. In *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability*, L. M. Le Cam and J. Neyman (Eds.), Vol. 1. University of California Press, 281–297.
- [41] E. J. Mishan. 1967. Pareto Optimality and the Law. *Oxford Econ. Pap.* 19, 3 (11 1967), 255–287.
- [42] Deepak Mittal, Shweta Bhardwaj, Mitesh M. Khapra, and Balaraman Ravindran. 2018. Recovering from random pruning: On the plasticity of deep convolutional neural networks. In *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*. 848–857.
- [43] Wei Niu, Xiaolong Ma, Sheng Lin, Shihao Wang, Xuehai Qian, Xue Lin, Yanzhi Wang, and Bin Ren. 2020. PatDNN: Achieving real-time DNN execution on mobile devices with pattern-based weight pruning. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20)*. Association for Computing Machinery, New York, NY, 907–922. <https://doi.org/10.1145/3373376.3378534>
- [44] Nvidia. 2020. CNMem Library: Simple Library to Help the Deep Learning Frameworks Manage CUDA Memory. Retrieved September 20, 2020 from <https://github.com/NVIDIA/cnmemo>.
- [45] Nvidia. 2020. CUDA Runtime API: CUDA Toolkit Documentation. Retrieved September 20, 2020 from <https://docs.nvidia.com/cuda/cuda-runtime-api>.
- [46] NVIDIA. 2020. CUDA Samples. Retrieved September 20, 2020 from <https://docs.nvidia.com/cuda/cuda-samples/index.html>.
- [47] NVIDIA. 2020. NVIDIA Ampere Architecture. Retrieved September 20, 2020 from <https://www.nvidia.com/en-us/data-center/nvidia-ampere-gpu-architecture/>.
- [48] OpenAI. 2020. AI and Compute. Retrieved September 20, 2020 from <https://openai.com/blog/ai-and-compute>.
- [49] PCI-SIG. 2020. 5.0 out of 5 Stars: PCI-SIG Member Companies Announce Support for the PCI Express 5.0 Specification. Retrieved September 20, 2020 from <https://pcisig.com/50-out-5-stars-pci-sig%C2%AE-member-companies-announce-support-pci-express%C2%AE-50-specification>.
- [50] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. 2020. Capuchin: Tensor-based GPU memory management for deep learning. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20)*. Association for Computing Machinery, New York, NY, 891–905. <https://doi.org/10.1145/3373376.3378505>
- [51] Yury Pisarchyk and Juhyun Lee. 2020. Efficient Memory Management for Deep Neural Net Inference. arXiv:2001.03288 [cs.LG]. Retrieved <https://arxiv.org/abs/2001.03288>.
- [52] Natalia Ponomareva, Soroush Radpour, Gilbert Hendry, Salem Haykal, Thomas Colthurst, Petr Mitrichev, and Alexander Grushetsky. 2017. TF boosted trees: A scalable TensorFlow based framework for gradient boosting. In *Proceedings of the European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD'17), Lecture Notes in Computer Science*, Vol. 10536. Springer, 423–427. https://doi.org/10.1007/978-3-319-71273-4_44
- [53] Alec Radford, Luke Metz, and Soumith Chintala. 2016. Unsupervised representation learning with deep convolutional generative adversarial networks. In *Proceedings of the 4th International Conference on Learning Representations (ICLR'16)*, Yoshua Bengio and Yann LeCun (Eds.).
- [54] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Austin, TX, 2383–2392. <https://doi.org/10.18653/v1/D16-1264>
- [55] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. 2016. VDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-49)*. IEEE Press, Article 18, 13 pages.

- [56] S. B. Shriram, Anshuj Garg, and Purushottam Kulkarni. 2019. Dynamic memory management for gpu-based training of deep neural networks. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS'19)*. 200–209.
- [57] M. Schuster and K. K. Paliwal. 1997. Bidirectional recurrent neural networks. *Trans. Sig. Proc.* 45, 11 (November 1997), 2673–2681. <https://doi.org/10.1109/78.650093>
- [58] Alex Sherstinsky. 2020. Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network. *Physica D: Nonlinear Phenomena* 404 (2020), 13230.
- [59] Karen Simonyan and Andrew Zisserman. 2015. Very deep convolutional networks for large-scale image recognition. In *Proceedings of the International Conference on Learning Representations*.
- [60] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alexander A. Alemi. 2017. Inception-v4, inception-ResNet and the impact of residual connections on learning. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence (AAAI'17)*. AAAI Press, 4278–4284.
- [61] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2015. Going deeper with convolutions. In *Proceedings of the Computer Vision and Pattern Recognition (CVPR'15)*.
- [62] Qiao Tian, Zewang Zhang, Heng Lu, Ling-Hui Chen, and Shan Liu. 2020. FeatherWave: An efficient high-fidelity neural vocoder with multi-band linear prediction. In *Proceedings of the IEEE Conference of the International Speech Communication Association (Interspeech'20)*. 195–199. <https://doi.org/10.21437/Interspeech.2020-1156>
- [63] A. Topa, L. N. Vicente, C. R. Paiva, and A. Barbosa. 1998. Application of the least squares method to the analysis of planar waveguides for integrated optics. In *Optimization*, Vol. 1. 63–63.
- [64] Mohamed Wahib, Haoyu Zhang, Truong Thao Nguyen, Aleksandr Drozd, Jens Domke, Lingqi Zhang, Ryousei Takano, and Satoshi Matsuoka. 2020. Scaling distributed deep learning workloads beyond the memory capacity with KARMA. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'20)*. IEEE.
- [65] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. Superneurons: Dynamic GPU memory management for training deep neural networks. In *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'18)*. Association for Computing Machinery, New York, NY, 41–53. <https://doi.org/10.1145/3178487.3178491>
- [66] Xueying Wang, Guangli Li, Xiao Dong, Jiansong Li, Lei Liu, and Xiaobing Feng. 2020. Accelerating deep learning inference with cross-layer data reuse on GPUs. In *Proceedings of the International European Conference on Parallel and Distributed Computing (Euro-Par'20)*, Maciej Malawski and Krzysztof Rzadca (Eds.). Springer International Publishing, Cham, 219–233.
- [67] Peter Weiner. 1973. Linear pattern matching algorithms. In *Proceedings of the 14th Annual Symposium on Switching and Automata Theory (SWAT'73)*. IEEE Computer Society, 1–11. <https://doi.org/10.1109/SWAT.1973.13>
- [68] Yidi Wu, Kaihao Ma, Xiao Yan, Zhi Liu, Zhenkun Cai, Yuzhen Huang, James Cheng, Han Yuan, and Fan Yu. 2022. Elastic deep learning in multi-tenant GPU clusters. *IEEE Trans. Parallel Distrib. Syst.* 33, 1 (2022), 144–158. <https://doi.org/10.1109/TPDS.2021.3064966>
- [69] Donglin Yang and Dazhao Cheng. 2020. Efficient GPU memory management for nonlinear DNNs. In *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing (HPDC'20)*. Association for Computing Machinery, New York, NY, 185–196. <https://doi.org/10.1145/3369583.3392684>
- [70] Junzhe Zhang, Sai-Ho Yeung, Yao Shu, Bingsheng He, and Wei Wang. 2019. Efficient memory management for GPU-based deep learning systems. arXiv:1903.06631. Retrieved from <http://arxiv.org/abs/1903.06631>.
- [71] Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V. Le. 2018. Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR'18)*. IEEE Computer Society, 8697–8710. <https://doi.org/10.1109/CVPR.2018.00907>

Received June 2021; revised February 2022; accepted May 2022